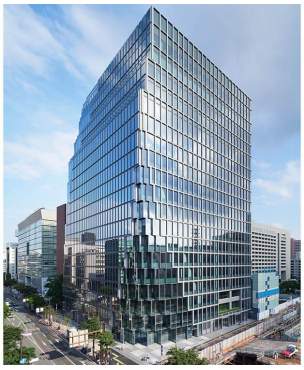
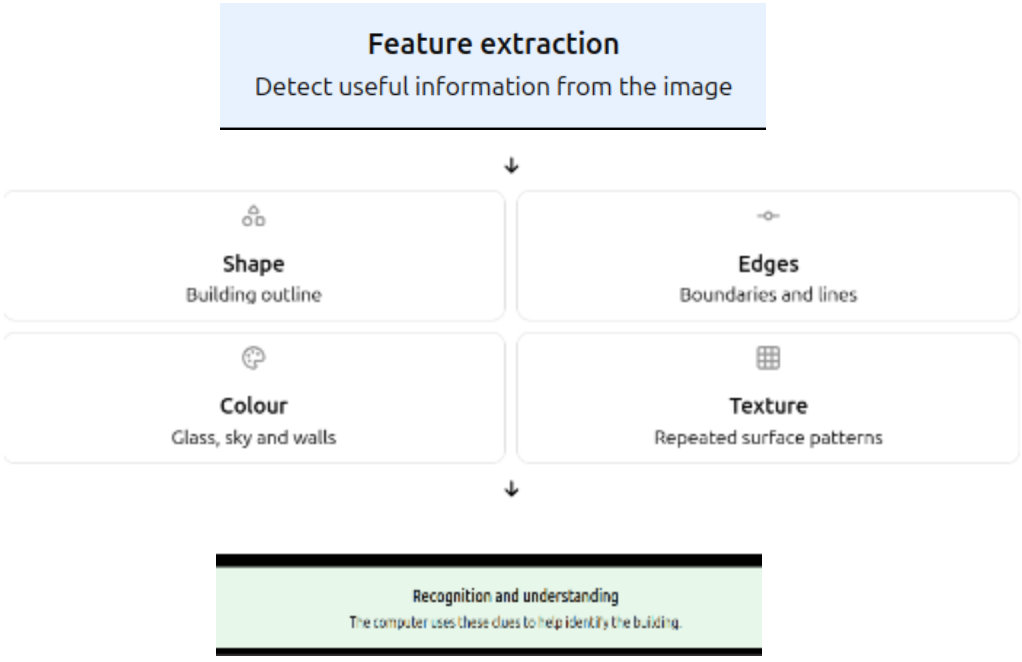
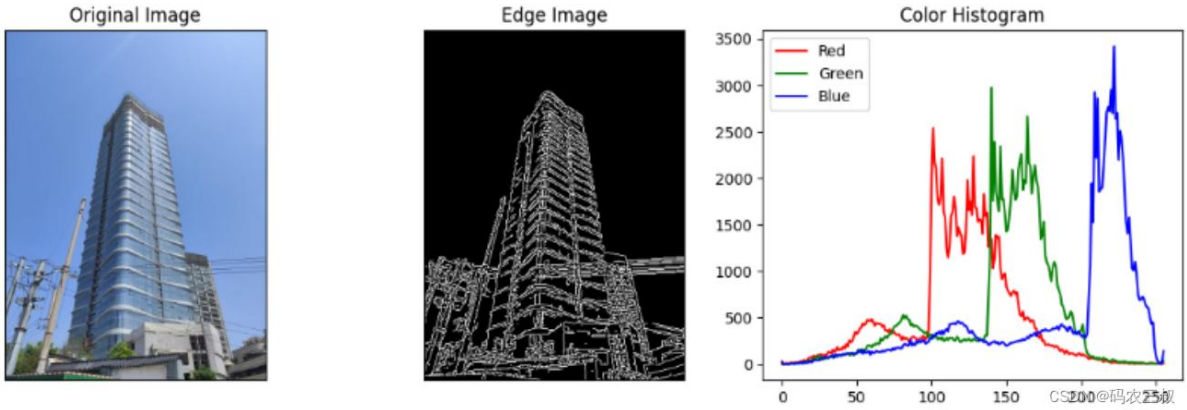
What is an image feature?

An image feature is a specific element or characteristic extracted from an image that provides meaningful information about its content.





FIVE important types of image features

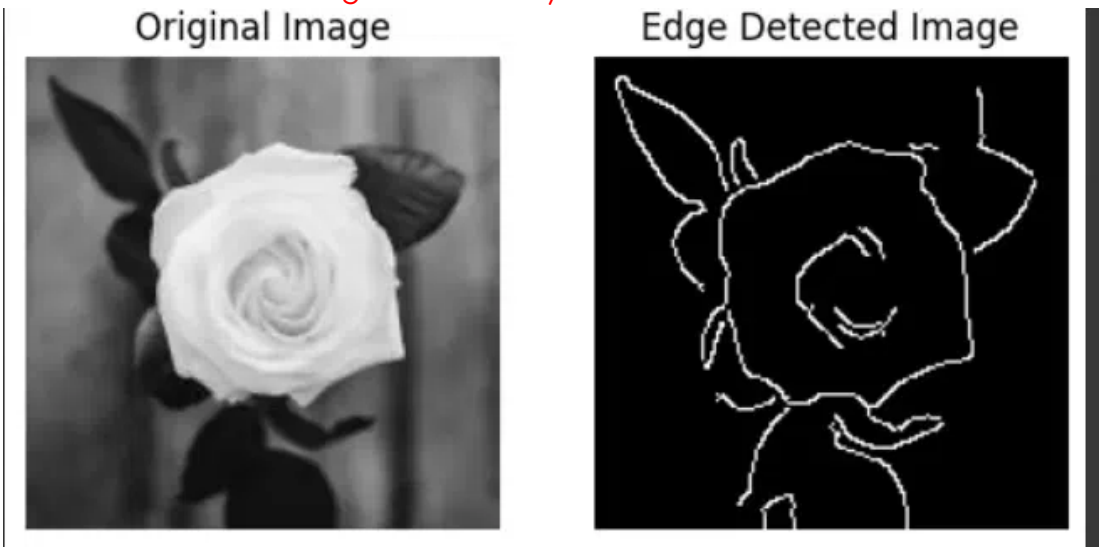


1. **Edge features**

An edge is a boundary where the image brightness or colour changes noticeably.

Example: The outline separating a building from the sky.

#NOTE: Remember: Edge = boundary.



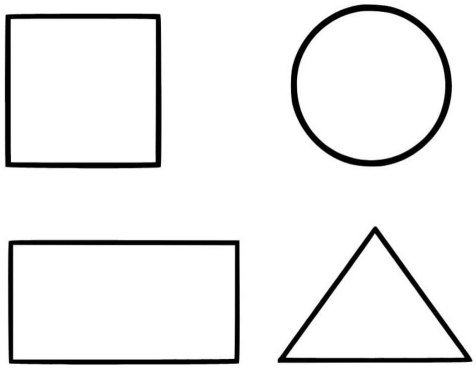


2. **Texture features**

Texture describes the surface pattern of an image, such as smoothness, roughness, or repetition.

Example: Brick walls have repeated patterns, while glass often appears smoother.

#NOTE : Remember: Texture = surface pattern.

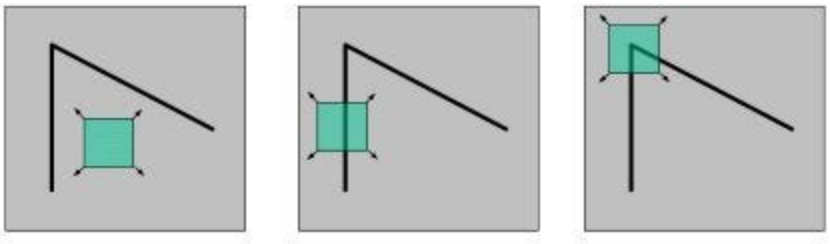


3. **Shape features**

These describe an object's outline or geometry, including its length, width, area, and general form.

Example: A computer can use rectangular shapes as one clue for identifying a **window**.

#NOTE : Remember: Shape = form or outline.



"flat" region:
no change in all directions

"edge":
no change along the edge direction

"corner":
significant change in all directions

4. Corner Features : Identity distinctive points where two or more edges intersect or where there is a significant intensity change in multiple directions

#NOTE: Remember: Corner= significant colour change in all direction.



(a) (b)

5. Colour features

These describe the colours present in an image and how they are distributed.

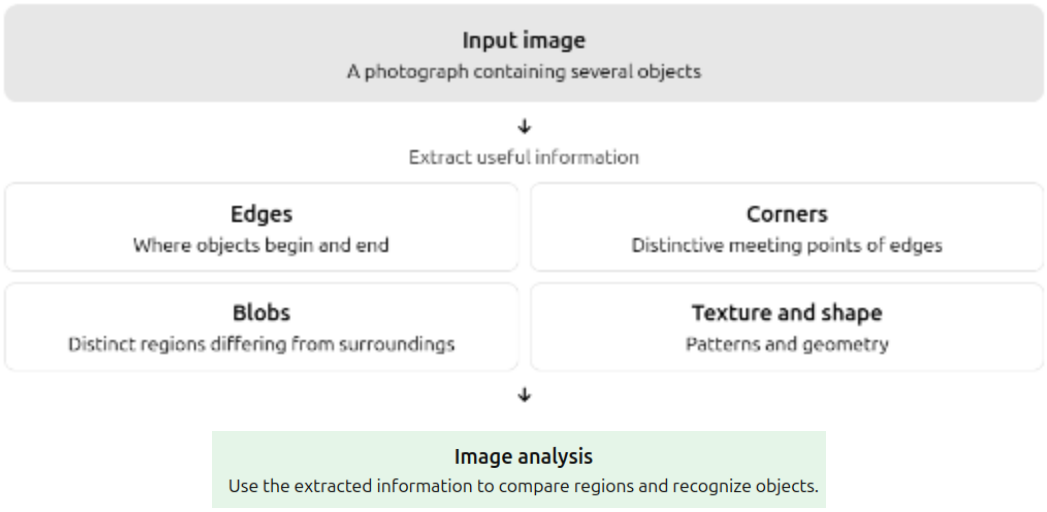
Example: Green pixels can help distinguish grass from a grey road.

#NOTE: Remember: Colour = visual colour information.

Why are image features important?

Image features help a computer analyse an image without treating every pixel as equally meaningful.

For example, when identifying a building, the computer may use its edges, shape, colour, and surface patterns together.



Applications

Image features are useful in many computer vision tasks:

- Object recognition: Identifying objects in photographs.
- Image matching: Finding similar objects or regions in different images.
- Image analysis: Examining boundaries, patterns, and shapes.
- Computer vision systems: Helping machines interpret visual information.

What is convolution?

Convolution is an operation in which a small matrix called a kernel or filter moves across an input image, performs element-wise multiplication with the image values, and sums the results to produce a **feature map**.

The three main things

1. Input image (A)

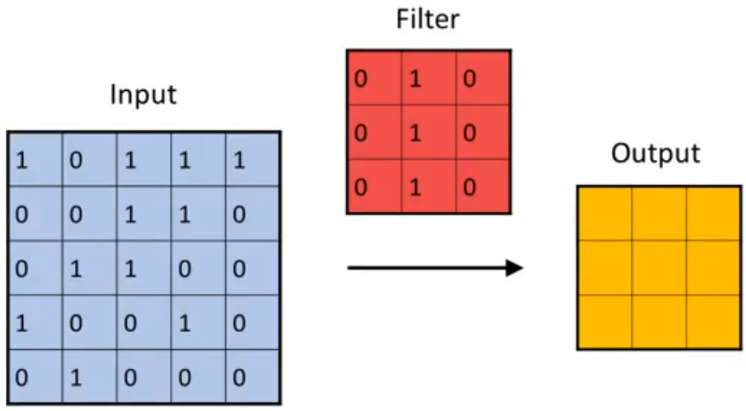
The original image represented as numbers in a matrix. Each number represents a pixel's intensity or colour value.

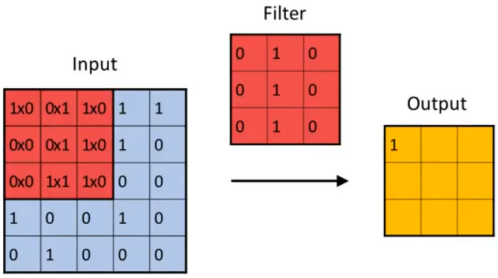
2. Kernel or filter (B)

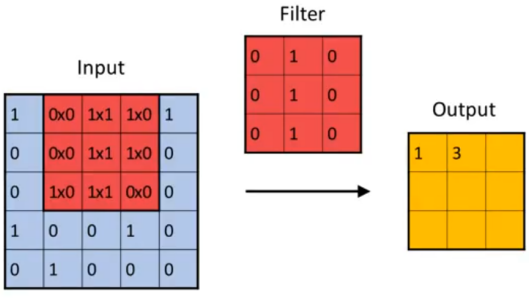
A small matrix, such as 3×3, that detects a particular pattern in the image.

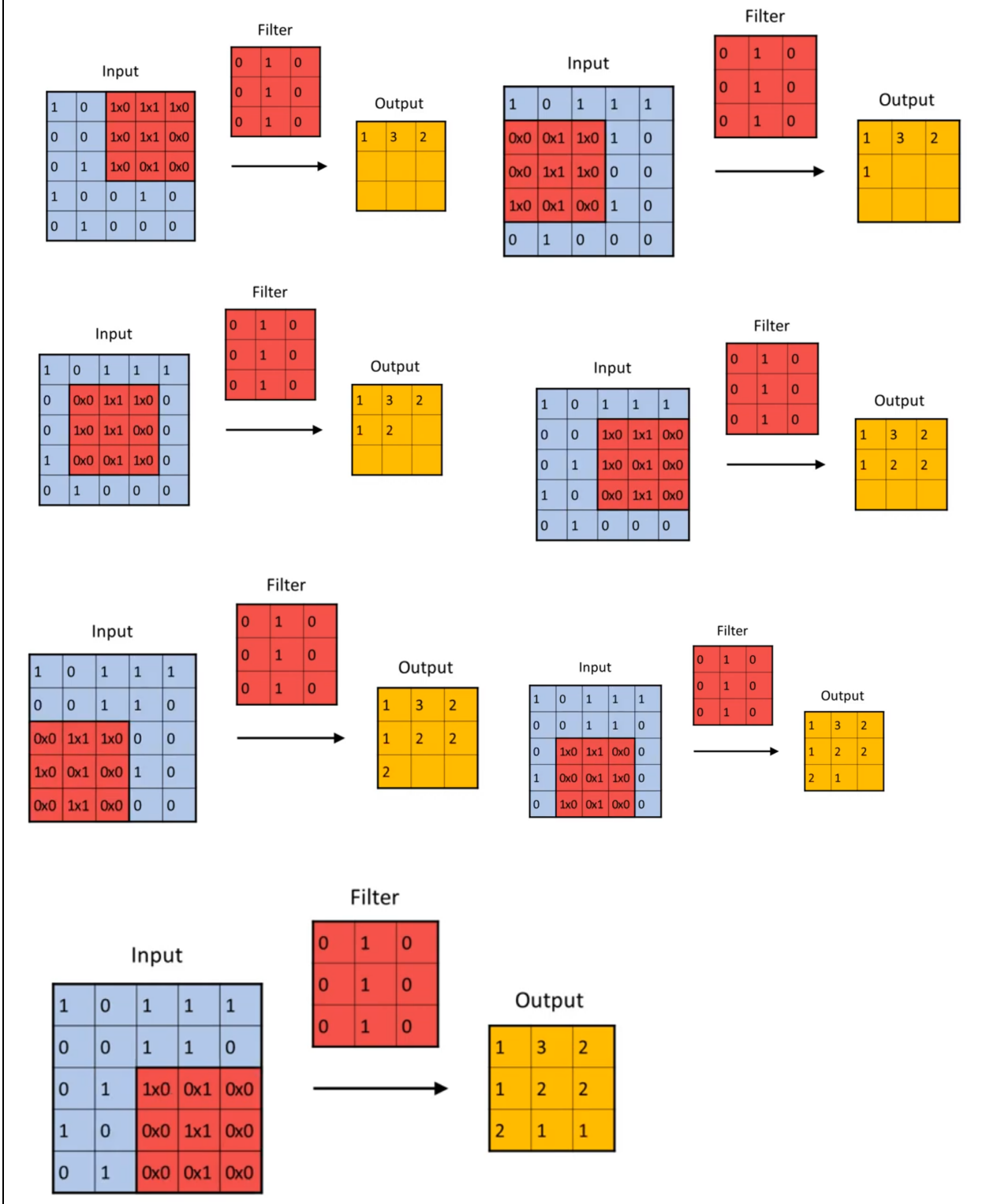
3. Feature map (C)

The output matrix produced by applying the kernel across the image. It indicates where the filter detects its pattern.









(matrix multiplication , as per you guys it's not in your syllabus, no stress)

What happens to the image?

For a square input of size N×N and a square kernel of K×K, using stride 1 and no padding.

Output size = (N−K+1)×(N−K+1)

- N = input image width or height.
- K = kernel width or height.
- Stride = number of pixels the kernel moves at each step.
- Padding = extra pixels added around the image boundary.

What is a kernel, exactly?

A kernel is a small matrix of numbers that examines a small area of the image at a time.

There are two different situations.

Situation A: Traditional image processing

A human programmer can manually choose a kernel designed for a specific task, such as sharpening an image or detecting edges.

Here, the numbers are chosen by the programmer.

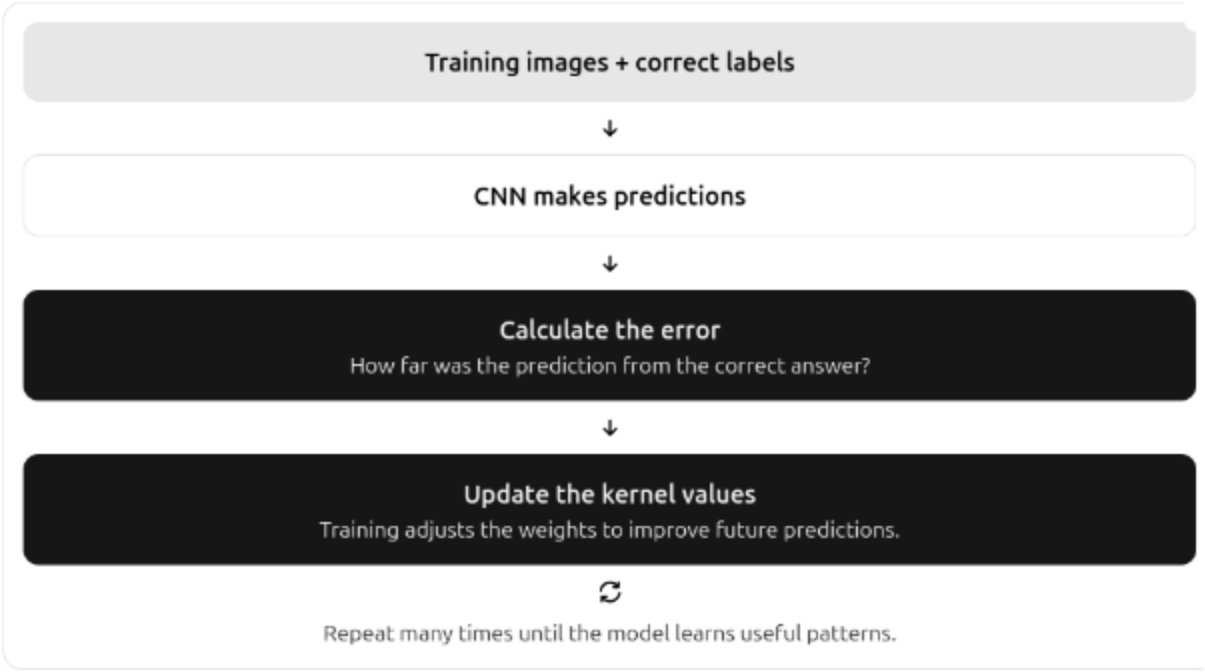
Situation B: CNN during training

The kernel's numbers are usually learnable parameters. The network starts with initial values, typically set automatically, and adjusts them during training using examples and a learning algorithm.

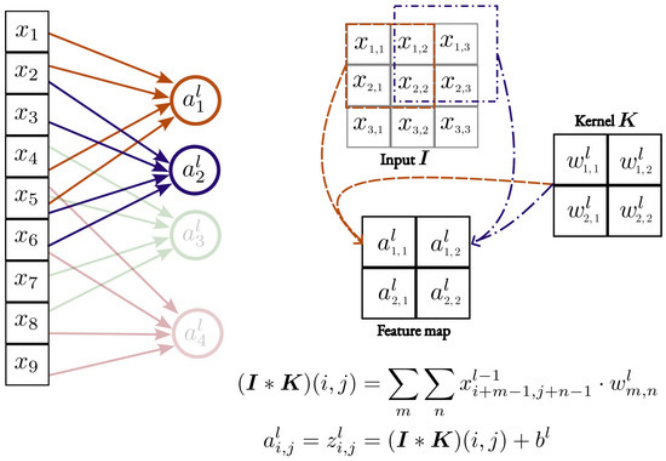
Here, the network learns useful filters from data rather than requiring a human to specify every edge-detection filter.

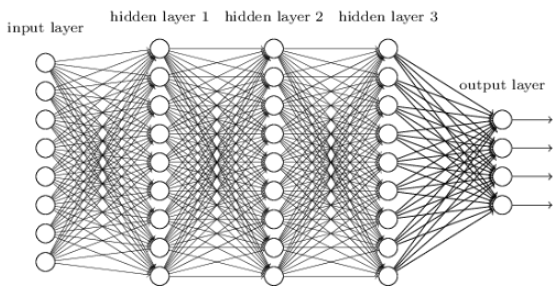
How does a CNN learn the kernel?

Imagine showing thousands of building photographs to a CNN, with labels telling it what each image contains.



This adjustment usually involves backpropagation and gradient descent which is in your btech syllabus. You don't need to master those to understand what a kernel is.





<<<<< adjusting kernel weight while training with images >>>>>>

What exactly is a feature map??

Forget the word for a moment. Imagine you have a small filter that searches for vertical edges in an image.

You slide it over the image. At every position, it calculates a number telling you how strongly the filter responds to that small region.

The collection of all those output numbers is called a feature map.

our previous feature map

Output

1	3	2
1	2	2
2	1	1



Why does pooling happen? Isn't the feature map already useful?

The feature map contains useful information, so why reduce it?

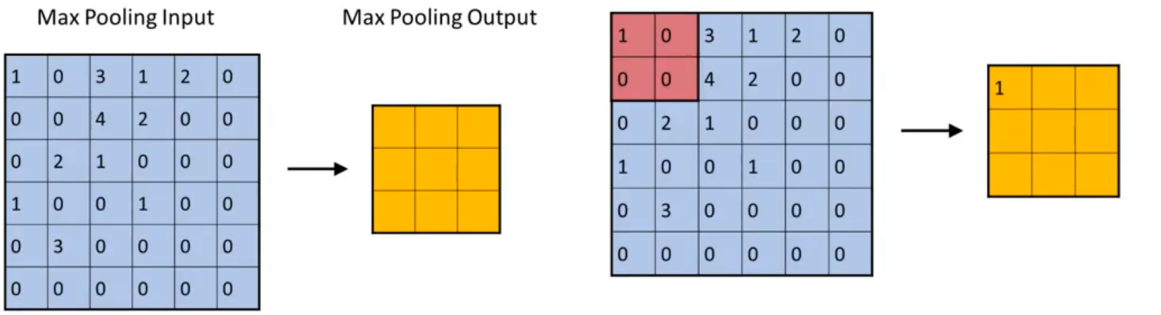


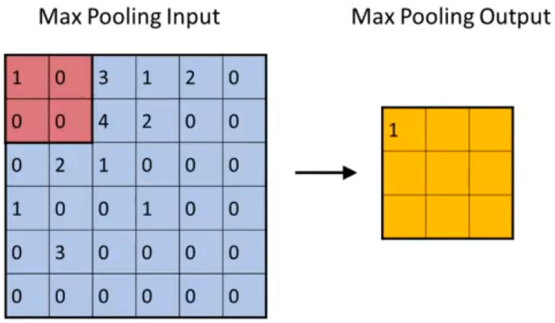
Imagine you are studying a huge map of a city. You don't need every tiny detail to answer the question, "Which direction is the railway station?" You may only need the important landmarks.

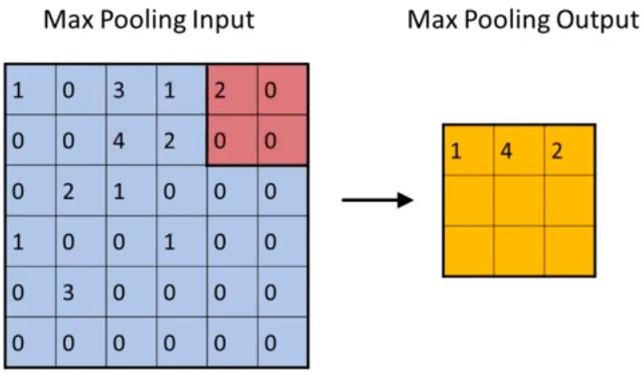
Pooling reduces the spatial size of a feature map so later layers have fewer values to process.

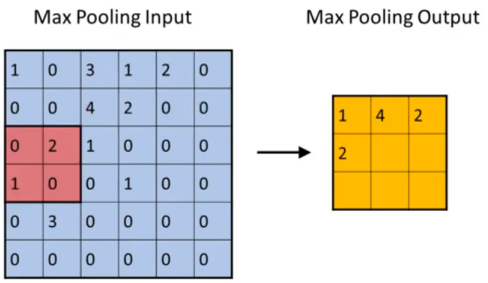
It is a form of summarising information, not the same thing as learning a kernel.

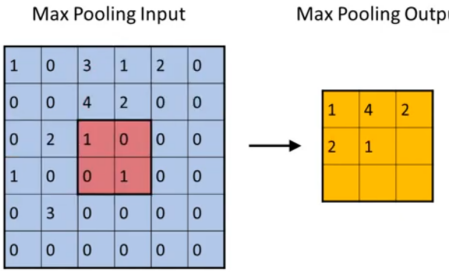
Movie time :

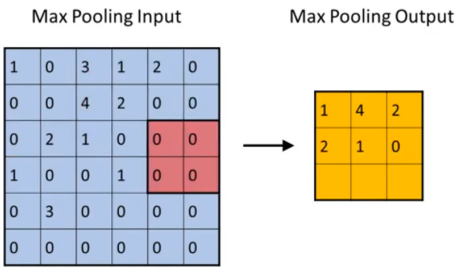






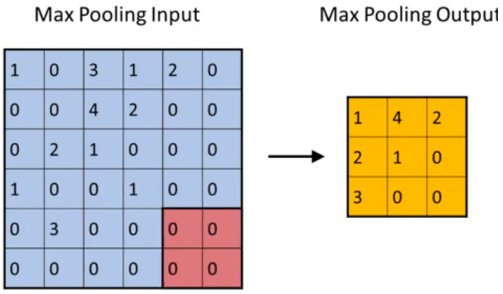






Your turn ?___>

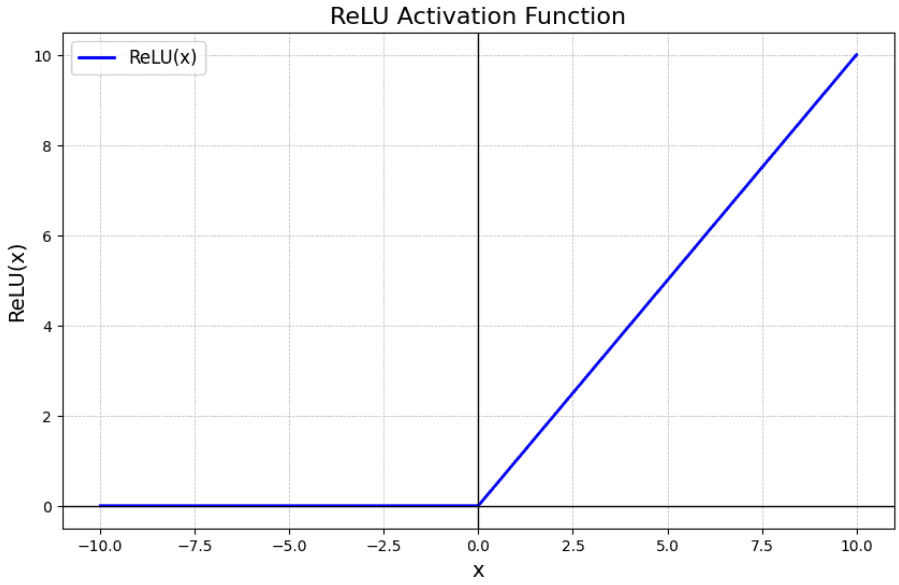




(ReLU)

ReLU stands for Rectified Linear Unit, a popular activation function used in artificial neural networks and deep learning models.

$$f(x) = \max(0, x)$$



Why do we use ReLU?

1. It introduces non-linearity. This allows a neural network to learn more complex

relationships instead of being limited to combinations of linear operations.

2. It removes negative activations. Negative values become zero, while positive values remain available for further processing.

3. It is simple to compute. It only requires comparing the input with zero.

#NOTE: ReLU does not automatically remove every unimportant feature. It specifically replaces negative values with zero; whether a feature is useful depends on the network and the task.

What is this Relu or ilu?

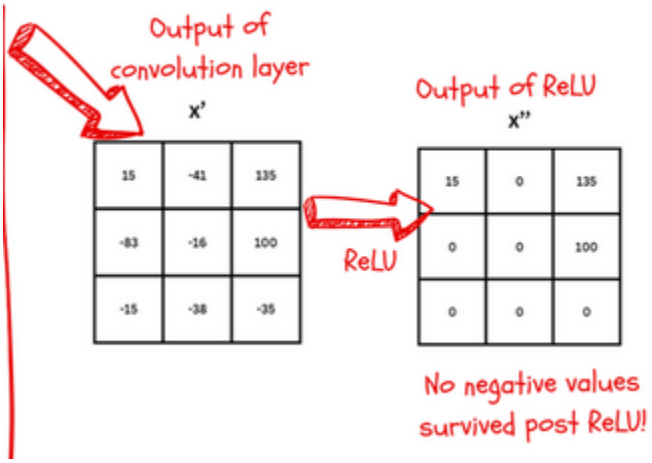


Imagine convolution has just finished

Your CNN takes a building image and applies a kernel to it. The kernel performs calculations, multiplying numbers and adding the results.

Those calculations produce a feature map.

But here's the important part: convolution can produce both positive and negative numbers



But why do we need ReLU at all?

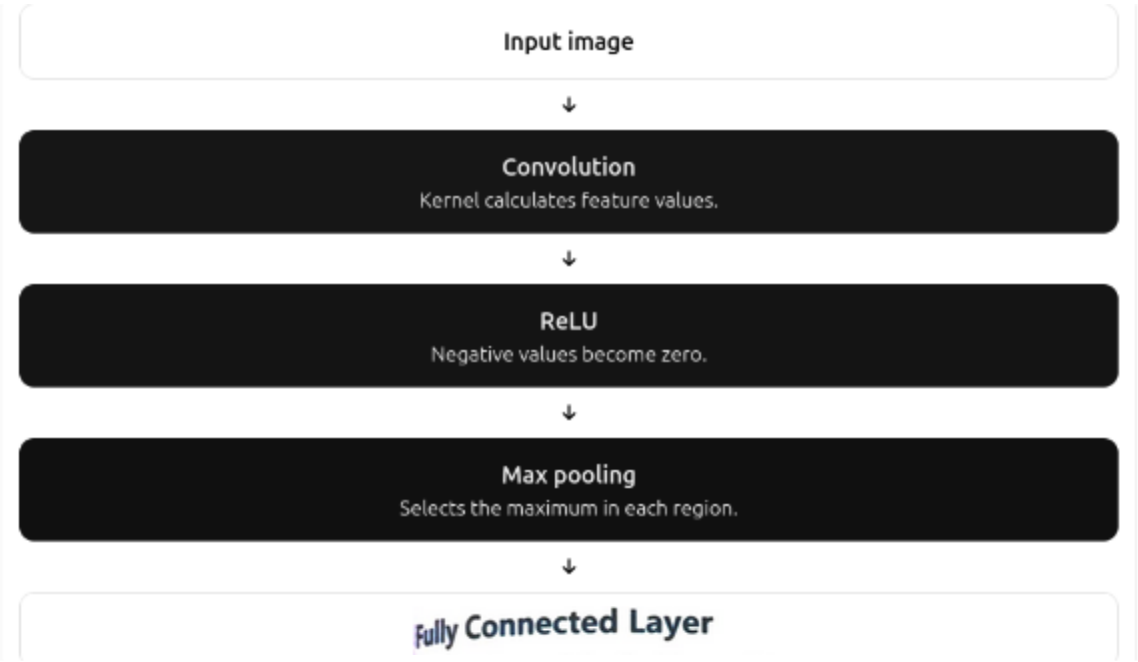
Think of CNN as a team.

- Convolution calculates responses to patterns.
- ReLU introduces non-linearity, allowing the network to learn more complex relationships.
- Pooling reduces the size of the feature map.

ReLU isn't simply a step to make the numbers smaller. Its most important mathematical role is introducing non-linearity.

Without nonlinear activation functions, stacking ordinary linear layers would still behave like a single linear transformation, limiting what the network can learn.

The final architecture :



Fully Connected Layer

This is the last and the final layer of the Convolutional Neural Network. After the features of the input image are extracted by the convolution layers and downsampled by the pooling layers, their output is a 3-dimensional matrix which is flattened into a vector of values. These values of the single vector represent a specific feature of a specific label and are redirected to fully connected layers to predict the final outputs of the network. This helps in classifying an image into a specific label based on the probability of the input being in a specific class. For example, the input image with a beak belongs to the category of a bird.

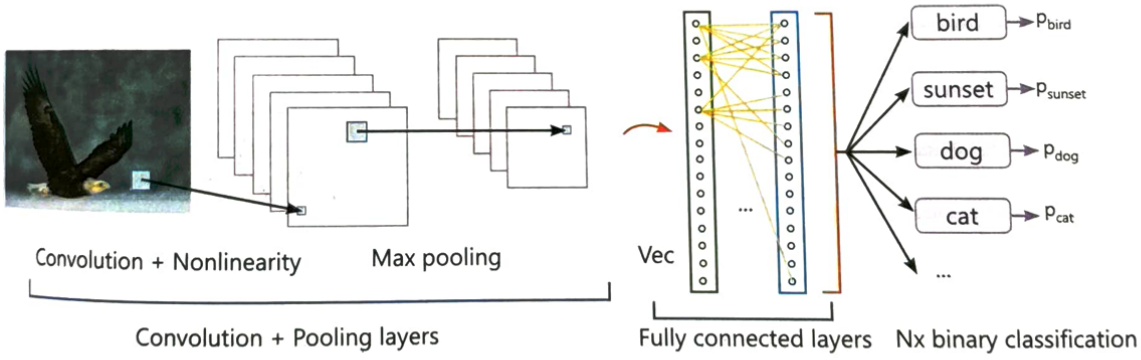
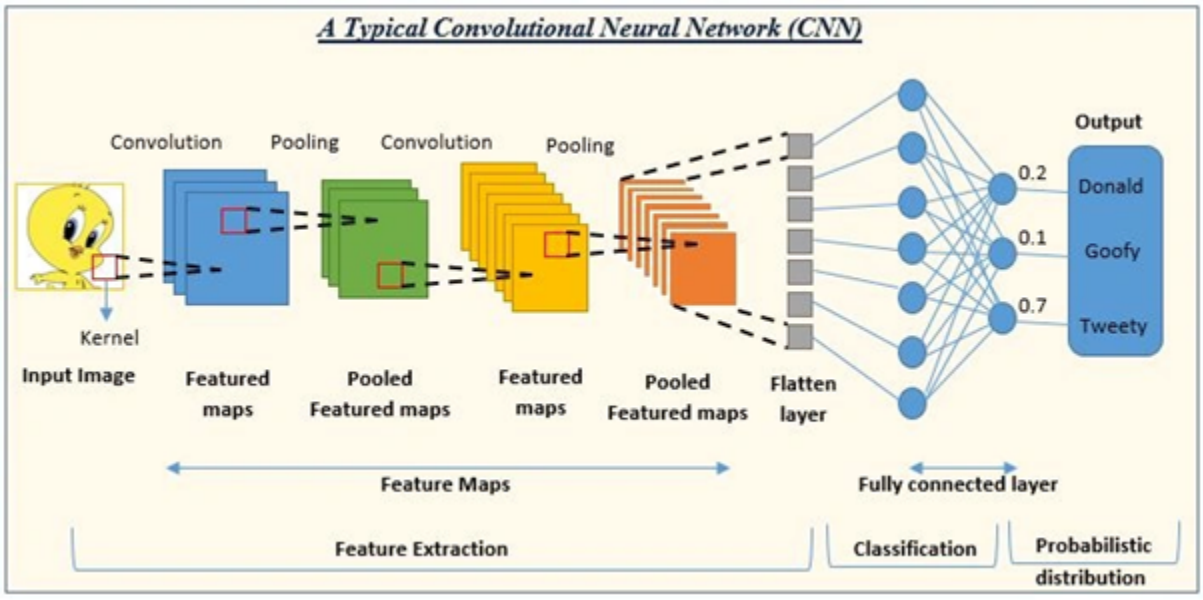


Image Classification Using CNN



Testing CNN

To test a CNN, you typically need to perform the following steps:

1. **Data Preparation:** Collect a dataset suitable for your specific task, such as image classification or object detection. Split the dataset into training, validation, and test sets.
2. **Model Selection:** Choose a CNN architecture suitable for your task, such as VGG, ResNet, or Inception. You may also consider using pre-trained models to save time and resources.
3. **Model Training:** Train the CNN on the training dataset using an appropriate loss function and optimisation algorithm. Monitor the model's performance on the validation set during training to avoid overfitting.
4. **Evaluation:** After training, evaluate the CNN's performance on the test dataset to assess its generalisation ability. The common evaluation metrics for image classification tasks include accuracy, precision, recall and F1 score.

5. **Fine-tuning:** If the performance of the CNN is not satisfactory, you may fine-tune the model by adjusting hyper parameters, changing the model architecture, or using data augmentation techniques.
6. **Deployment:** Once you are satisfied with the model's performance, you can deploy it to make predictions on new and unseen data.

CNN_Visual_Lab_MNIST.ipynb : visual output

The End :& The Last Note :

